6.7960 MIT deep Learning and generative AI

MIT courses-> Why of everything.. not just saying convention is.. even coursera..

Course Overview

Apply each concept to a real project.. leading to a final thesis with practice.

Description: Fundamentals of deep learning, including both theory and applications. Topics include neural net architectures (MLPs, CNNs, RNNs, graph nets, transformers), geometry and invariances in deep learning, backpropagation and automatic differentiation, learning theory and generalization in high-dimensions, and applications to computer vision, natural language processing, and robotics.
Pre-requisites: 18.05 and (6.3720, 6.3900, or 6.C01)
Note: This course is appropriate for advanced undergraduates and graduate students, and is 3-0-9 units.

Readings

Readings labeled [Vision] are from *Foundations of Computer Vision* by Antonio Torralba, Phillip Isola, and William T. Freeman. (MIT Press, 2024. ISBN: 9780262048972.) The book is available online under a CC BY-NC-ND license.
Session 1: Introduction to Deep Learning
Required readings:
- Notation for this course
- [Vision] Chapter 12: Neural Networks.

Optional readings:
- [Vision] Chapter 13: Neural Networks as Distribution Transformers

Session 2: How to Train a Neural Net
Required readings:
- [Vision] Chapter 10: Gradient-Based Learning Algorithms
- [Vision] Chapter 14: Backpropagation

Session 3: Approximation Theory
No required readings.
Optional readings:
- Deep learning theory lecture notes sections 2 and 5

Session 4: Architectures: Grids
Required readings:
- [Vision] Chapter 24: Convolutional Neural Nets

Session 5: Architectures: Graphs

Required readings:

- Hamilton, William. *Graph Representation Learning*, chapter 5 (mainly focus on the content in section 5.1)

Optional readings:

- Xu, Keyulu, Weihua Hu, et al. "How Powerful Are Graph Neural Networks?" arXiv preprint arXiv:1810.00826 (2018).
- Sanchez-Lengeling, Benjamin, Emily Reif, et al. "A Gentle Introduction to Graph Neural Networks." Distill, 2021.

Session 6: Generalization Theory

No required readings.

Optional readings:

- Zhang, Chiyuan, Samy Bengio, et al. "Understanding Deep Learning Requires Rethinking Generalization." *arXiv preprint arXiv:1611.03530* (2016).
- Belkin, Mikhail, Daniel Hsu, et al. "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off." *Proceedings of the National Academy of Sciences* 116, no. 32 (2019): 15849–15854.

Session 7: Scaling Rules for Optimization

Required readings:

- 13.7 General Steepest Descent

Session 8: Architectures: Transformers

Required readings:

- [Vision] Chapter 26: Transformers (Note that this reading focuses on examples from vision, but you can apply the same architecture to any kind of data.)

Session 9: Hacker's Guide to Deep Learning

Optional readings:

- A Recipe for Training Neural Networks
- Rules of Machine Learning: Best Practices for ML Engineering (PDF)

Session 10: Architectures: Memory

Required readings:

- [Vision] Chapter 25: Recurrent Neural Networks

Optional readings:

- Deep Learning Recurrent Networks: Stability Analysis and LSTMs (PDF)

Session 11: Representation Learning: Reconstruction-Based

Required readings:

- [Vision] Chapter 30: Representation Learning

Optional readings:

- Bengio, Yoshua, Aaron Courville, and Pascal Vincent. "Representation Learning: A Review and New Perspectives." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35, no. 8 (2013): 1798–1828.

Session 12: Representation Learning: Similarity-Based

Required readings:

- Continue with session 11 readings

Optional readings:

- Wang, Tongzhou, and Phillip Isola. "Understanding Contrastive Representation Learning Through Alignment and Uniformity on the Hypersphere." In *International Conference on Machine Learning*, pp. 9929–9939. PMLR, 2020.

Session 13: Representation Learning: Theory

No required readings.

Optional readings:

- Cho, Youngmin, and Lawrence Saul. "Kernel Methods for Deep Learning." *Advances in Neural Information Processing Systems 22* (2009).
- Lee, Jaehoon, Yasaman Bahri, et al. "Deep Neural Networks as Gaussian Processes." *arXiv preprint arXiv:1711.00165* (2017).

Session 14: Generative Models: Basics

Required readings:

- [Vision] Chapter 32: Generative Models

Session 15: Generative Models: Representation Learning Meets Generative Modeling
Required readings:
- [Vision] Chapter 33: Generative Modeling Meets Representation Learning

Optional readings:
- Kingma, Diederik P., and Max Welling. "Auto-Encoding Variational Bayes." *arXiv preprint arXiv:1312.6114* (2013).

Session 16: Generative Models: Conditional Models
No required readings.
Optional readings:
- [Vision] Chapter 34: Conditional Generative Models

Session 17: Generalization: Out-of-Distribution (OOD)
Required readings:
- Mądry, Aleksander, and Ludwig Schmidt. "A Brief Introduction to Adversarial Examples." gradient science, July 6, 2018.
- Mądry, Aleksander, Ludwig Schmidt, and Dimitris Tsipras. "Training Robust Classifiers (Part 1)." gradient science, July 11, 2018.

Optional readings:
- Koh, Pang Wei, Shiori Sagawa, et al. "Wilds: A Benchmark of In-the-Wild Distribution Shifts." In *International Conference on Machine Learning*, pp. 5637–5664. PMLR, 2021.
- Geirhos, Robert, Jörn-Henrik Jacobsen, et al. "Shortcut Learning in Deep Neural Networks." *Nature Machine Intelligence* 2, no. 11 (2020): 665–673.
- Tsipras, Dimitris, Shibani Santurkar, et al. "From Imagenet to Image Classification: Contextualizing Progress on Benchmarks." In *International Conference on Machine Learning*, pp. 9625–9635. PMLR, 2020.
- Xiao, Kai, Logan Engstrom, et al. "Noise or Signal: The Role of Image Backgrounds in Object Recognition." *arXiv preprint arXiv:2006.09994* (2020).
- Xu, Keyulu, Mozhi Zhang, et al. "How Neural Networks Extrapolate: From Feedforward to Graph Neural Networks." *arXiv preprint arXiv:2009.11848* (2020).

Session 18: Transfer Learning: Models
Required readings:
- [Vision] Chapter 37: Transfer Learning and Adaptation

Optional readings:
- Farahani, Abolfazl, Sahar Voghoei, et al. "A Brief Review of Domain Adaptation." *Advances in Data Science and Information Engineering: Proceedings from ICDATA 2020 and IKE 2020* (2021): 877–894.
- Kay, Justin, Timm Haucke, et al. "Align and Distill: Unifying and Improving Domain Adaptive Object Detection." *arXiv preprint arXiv:2403.12029* (2024).

Session 19: Transfer Learning: Data
Required readings:
- Continue with session 19 readings.

Session 20: Scaling Laws
Required readings:
- Kaplan, Jared, Sam McCandlish, et al. "Scaling Laws for Neural Language Models." *arXiv preprint arXiv:2001.08361* (2020).

Optional readings:
- Hoffmann, Jordan, Sebastian Borgeaud, et al. "Training Compute-Optimal Large Language Models." *arXiv preprint arXiv:2203.15556* (2022).
- Sharma, Utkarsh, and Jared Kaplan. "A Neural Scaling Law from the Dimension of the Data Manifold." *arXiv preprint arXiv:2004.10802* (2020).
- Sorscher, Ben, Robert Geirhos, et al. "Beyond Neural Scaling Laws: Beating Power Law Scaling via Data Pruning." *Advances in Neural Information Processing Systems* 35 (2022): 19523–19536.
- McCandlish, Sam, Jared Kaplan, et al. "An Empirical Model of Large-Batch Training." *arXiv preprint arXiv:1812.06162* (2018).

Session 21: Large Language Models
No required readings.
Optional readings:

- Kojima, Takeshi, Shixiang Shane Gu, et al. "Large Language Models Are Zero-Shot Reasoners." *Advances in Neural Information Processing Systems* 35 (2022): 22199–22213.

Session 22: AI for Musical Creativity

No required readings.

Session 23: Metrized Deep Learning

No required readings.

Optional readings:

- Bernstein, Jeremy, and Laker Newhouse. "Modular Duality in Deep Learning." *arXiv preprint arXiv:2410.21265* (2024).
- Flynn, Thomas. "The Duality Structure Gradient Descent Algorithm: Analysis and Applications to Neural Networks." *arXiv preprint arXiv:1708.00523* (2017).
- Large, Tim, Yang Liu, et al. "Scalable Optimization in the Modular Norm." *Advances in Neural Information Processing Systems* 37 (2024): 73501–73548.

Session 24: Inference Methods for Deep Learning

No required readings.

Optional readings:

- Sun, Yu, Xiaolong Wang, et al. "Test-Time Training with Self-Supervision for Generalization Under Distribution Shifts." In *International Conference on Machine Learning*, pp. 9229–9248. PMLR, 2020.
- Zelikman, Eric, Yuhuai Wu, et al. "Star: Bootstrapping Reasoning with Reasoning." *Advances in Neural Information Processing Systems* 35 (2022): 15476–15488.

Session 25: Efficient Policy Optimization Techniques for LLMs

No required readings.

- Materials

- **Readings will come from a variety of sources and will be posted on the schedule each week.**
- Some readings are derived from the course textbook, which can be found for free online: Foundations of Computer Vision.
- The best textbook devoted entirely to deep learning is probably Understanding Deep Learning, which is freely available online.
- Content from 6.390 Intro to ML can also be found here for those who want to brush up on ML concepts.

Schedule

Date	Topics	Speaker	Course Materials	Assignments	
Week 1	fracture date				
Thu 6/4	Course overview, introduction to deep neural networks and their basic building blocks	Sara Beery	slides optional readings: notation for this course neural networks		
Week 2					

Mon 6/8	PyTorch Tutorial: 4-5 PM, 32-123		pytorch tutorial colab		
Tue 6/9	How to train a neural net **+ details** *SGD, Backprop and autodiff, differentiable programming*	Sara Beery	slides required readings: gradient-based learning backprop	pset 1 out (solutions)	
Tue 6/9	PyTorch Tutorial: 3-4 PM, 2-190				
Wed 9/10	PyTorch Tutorial: 7-8 PM, 32-123				
Thu 9/11	Approximation theory **+ details** *How well can you approximate a given function by a DNN? We will explore various facets of this issue, from universal approximation to Barron's theorem. And does increasing the depth provably help for expressivity?*	Omar Khattab	slides		
Fri 9/12	PyTorch Tutorial: 11 AM -12 PM, 32-123				
Week 3					

Tue 6/16	Architectures: Grids **+ details** *This lecture will focus mostly on convolutional neural networks, presenting them as a good choice when your data lies on a grid.*	Sara Beery	slides required reading: CNNs		
Thu 6/18	Architectures: Memory and Sequence Modeling **+ details** *RNNs, LSTMs, memory, sequence models.*	Kaiming He	slides		
Week 4					
Tue 6/23	Architectures: Transformers **+ details** *Transformers. Three key ideas: tokens, attention, positional codes. Relationship between transformers and MLPs, GNNs graph neural networks, and CNNs -- they are all variations on the same themes!*	Sara Beery	slides reading: Transformers (note that this reading focuses on examples from vision but you can apply the same architecture to any kind of data)	pset 1 due pset 2 out (solutions)	

Thu 6/25	Generalization Theory	Omar Khattab	slides optional readings: Deep learning requires rethinking generalization Deep learning not so mysterious or different Data science at the singularity		
Week 5					
Tue 6/30	Representation Learning: Reconstruction-based **+ details** *Intro to representation learning, unsupervised and self-supervised learning, clustering, dimension reduction, autoencoders, and modern self-supevised learning with reconstruction losses.*	Kaiming He	slides		

Thu 7/2	Representation Learning: Similarity-based (aka Neural Information Retrieval) **+ details** *Information retrieval, contrastive learning (InfoNCE; hard negatives; KL distillation; self-supervised vs. supervised), sub-linear search and scaling tradeoffs (cross-encoders; bi-encoders; late*	Omar Khattab	slides optional readings: Contrastive Representation Learning Contextualized Late Interaction over BERT In Defense of Dual-Encoders		
Week 6					
Tue 7/7	Representation Learning and Information Theory	Kaiming He	slides	pset 2 due pset 3 out(1-2 weeks)	
Thu 7/9	Foundation models: pre-training	Omar Khattab	slides optional readings: Language Models are Few-Shot Learners SmolLM3; OLMo 2; Marin 8B		
Week 7	till here only, till wearing sling..				

Tue 7/14	Foundation models: scaling laws	Omar Khattab	slides optional readings: Scaling Laws for LLMs Training Compute-Optimal LLMs Emergent Abilities of LLMs Are Emergent Abilities of LLMs a Mirage?		
Thu 7/16	Generative models: basics	Kaiming He	slides	pset 3 due pset 4 out	
Week 8					
Tue 7/21	Midterm: 7:30 - 9:30 PM				
Thu 7/23	Generative models: VAE and GAN	Kaiming He	slides		
Week 9					
Tue 7/28	Foundation models: post-training	Omar Khattab	slides		
Thu 7/30	Generative models: Diffusion and Flows	Kaiming He	slides		
Week 10					
Tue 8/4	Generalization (OOD) **+ details** *Exploring model generalization out of distribution, with a focus on adversarial robustness and distribution shift*	Sara Beery	slides	pset 4 due pset 5 out	

Thu 8/6	Transfer learning: Models and Data **+ details** *Finetuning, linear probes, knowledge distillation,* ***generative models as data, domain adaptation, prompting***	Sara Beery	slides		
Week 11					
Tue 8/11	No class: Veterans Day				
Thu 8/13	Inference-time Algorithms	Omar Khattab	slides	pset 5 due	
Week 12					
Tue 8/18	Guest Lecture: Deep Learning that Improves Real-World Interactions	Rose E Wang (OpenAI)			
Thu 8/20	Evaluation **+ details** *Designing benchmarks, selecting metrics, and using hu****man input at inference***	Sara Beery	slides		
Week 13					
Tue 8/25	Applying Deep Learning to Your Problems	Kaiming He	slides		
Thu 8/27	No class: Thanksgiving Day				
Week 14					
Tue 9/2	Guest Lecture 2	Zongyi Li (MIT/NYU)			
Thu 9/4	Project office hours				
Week 15					

Tue 9/15	Guest Lecture 3	Jiajun Wu (Stanford)		project due	

Gold Standard

If your goal is to truly understand deep learning (not just run models), here's a resource you shouldn't ignore.

The MIT 6.7960 Deep Learning playlist from **MIT OpenCourseWare** (taught by Dr. **Sara Beery**, Dr. **Phillip Isola** and Prof. Jeremy Bernstein) is a complete university-level course designed to give you a rigorous and modern understanding of deep learning.

It goes far beyond surface-level tutorials and focuses on the theory, intuition, and architecture design behind today's most powerful AI systems.

Here's what you'll gain from it:

1. A deep understanding of core deep learning concepts such as:
   a) Neural network training and backpropagation
   b) Optimization techniques and loss landscapes
   c) Approximation theory
   d) CNNs for images and grid data
   e) RNNs for sequential data
   f) Transformers and attention mechanisms
   g) Graph Neural Networks

2. Strong intuition about:
   a) Why deep networks work (not just how)
   b) Model invariances and geometric structure
   c) Representation learning
   d) Generalization and transfer learning
   e) Failure modes and debugging strategies

3. Practical exposure to real-world deep learning thinking:
   a) Designing architectures from scratch
   b) Understanding trade-offs between models
   c) Translating theory into frameworks like PyTorch or TensorFlow

Here's how to use this resource effectively:

Step 1: Follow the lectures sequentially. This course is structured; skipping will break your understanding.

Step 2: Take notes on derivations and key ideas. This is not a passive course.

Step 3: Implement everything. Rebuild models (CNNs, transformers) from scratch in notebooks.

Step 4: Use the official assignments and materials on OCW. That's how you'll really learn.

Step 5: Apply each concept to a real project ..

Step 6: Revisit complex topics like optimization and transformers multiple times; you might not understand them on the first pass.

Access the playlist here: **https://lnkd.in/eHx3CkVN**

Access the full course materials here: **https://lnkd.in/eyDUeE4T**

This is not a beginner-friendly resource; but if you commit to it, it can move you from “using models” to actually understanding and building them.

Kyin fi

1. Hacker’s Guide
2. Outline
   - 1. Intro to Deep Learning
   - 2. How to train a neural net
   - 3. Approximation Theory
   - 4. Architectures for Grids
   - 5. Graph Neural Networks
   - 6. Neural Network Generalization
   - 7. Scaling Rules for Optimization
   - 8. Transformers
   - 9. Hacker’s guide to DL
   - 10. Memory and sequence modeling
   - 11. Representation Learning I
   - 12. Representation Learning II: Similarity-Based
   - 13. Representation Learning Theory
   - 14. Generative Models: Basics
   - 15. Generative Models and Representation Learning
   - 16. Conditional Generative Models
   - 17. Out-of-distribution Generalization
   - 18. Transfer Learning: Models
   - 19. Transfer Learning: Data
   - 20. Scaling Laws
   - 21. Large Language Models
   - 22. AI for Musicians
   - 23. Metrized Deep Learning
   - 24. Inference Methods for Deep Learning
   - 25. Efficient Policy Optimization for LLMs
   - 26. Project (AGAN)

Overview of foundational and state-of-the-art deep learning methods (as of Fall 2024). This semester’s content should also be posted on OCW.

Hacker’s Guide

Presented by Phillip Isola (Lecture 9). Here is a summary:
Become friends with every pixel.
- Look at the input.
- Look at (all aspects of) the training and output.

Look at the data.
- Histogram (distribution) of input features and targets. Want diverse data.
- Inspect the data right before calling model(data), for debugging.
- Make an inspect_data(X) function that prints type, shape, requires_grad, min/max, and mean/var, for debugging. See lecture notes.

Datasets
- Use data augmentation to get the invariances you want.
- Make data harder than it would be in reality with domain randomization. The goal is to have training data that mimics possible real world changes in the test data.
- Make your data hard enough to learn a lot from.
  Formula: max over data(min over param(loss(data, params)))
- Use big data: Lots of training pairs, high-dimensional inputs and outputs.
- Prediction gets easier with longer/more inputs.

Preprocessing
- Standardize all data, probably. x = (x - E[x]) / sqrt(Var[x]).
- Sanity check shapes with prime dimensional data.
- Check for unintended casts.

Model
- The simplest model/explanation that fits the data is the best model.
- Start with a standard, popular model. Popularity is more reliable than benchmarks.
- Transform your problem into a "solved" problem (like polynomial time reductions).
- Every piece of complexity you add has a cost.
- Avoid low dimensions (weird behavior). All variables should be >= 8.
- The easiest ways to get better performance: Scale your data, model, or compute.
- Once you get your system working, remove everything nonessential.

Defaults
- Image classification: https://pytorch.org/hub/pytorch_vision_resnet/
- Text generation: https://huggingface.co/models?pipeline_tag=text-generation
- Transform data into one-hot vectors. Use cross-entropy loss, Adam, and a transformer architecture. Use layer norm, not batch norm.

Code
- A lot of your code will be reshaping tensors. Consider using einops for readability.
- Copilots are great for boilerplate code, visuals, and syntax. Give it documentation.

Optimization
- Adam is good for prototyping. SGD is good for performance. Clip gradients for stability.
- Use a constant learning rate at first. Retune/schedule after everything else is settled.
- Don't tie your learning rate schedule to the number of epochs (makes it hard to compare learning curves). There are no epochs in the wild.
- Use exponential moving averages (EMA) for gradients.
- Make checkpoints of your model in case your computer crashes / the model breaks.

Evaluation
- Switch to evaluation mode using model.eval(), and check with print(model.training).
- Look at the output and loss curves to diagnose issues. Compare different size models.
- WandB and Tensorboard are your friends for logging/checkpointing.

Tuning
- Deep learning is analogous to cooking. Learn what each ingredient (data/architecture) and spice (augmentation) does. Find the right combo to solve your problem.
- Try extreme settings for a bit. If the model never diverges, your learning rate is too low.

Debugging
- Script the optimization/evaluation of your models. Use config files to manage them.
- Try to train your model on one datapoint, then a few, then the full set.
- Sanity check your loss against a reference value (ln(0.5) = -0.69).
- Use the Python debugger. To find NaNs, use torch.autograd.set_detect_anomaly(True).
- Common bugs: If out of memory, don't store gradients at inference time, and clear memory with del when needed. When timing code on GPUs, torch.cuda.synchronize().

Compute
- Only parallelize if your model works on one device. More hardware = more problems.
- Saturate your GPUs to 100% utilization.
- For speed, try torch.cudnn.benchmark, automatic mixed precision, and torch.compile().

Outline

Due to no cheatsheets, I'll include an outline of the lectures + a short summary of each. Refer to lecture notes or the Fall 2024 schedule for more details.

1. Intro to Deep Learning

Review of machine learning and neural nets. Intro to tensors and deep net generalization / the double-descent curve. PyTorch primer.

2. How to train a neural net

Review of gradient descent / SGD. Loss functions like GeLU. Computation graphs. Backpropagation details through chains, MLPs, and DAGs. Differentiable programming, software 2.0 using blocks optimized to a scalar cost. (Homework 1 covered neural net basics.)

3. Approximation Theory

Universal approximation theory of neural nets, proof details (Lipschitz continuous functions, approximate with hyperrectangles). Width vs. depth scaling, depth separation results example (to show that depth can scale better). Practical considerations for generalization: More parameters is better, depth of >6 layers seems to have negligible effect.

4. Architectures for Grids

Building better architectures with good inductive biases (built-in assumptions for the right answer / generalization). Convolutional neural networks (CNNs) providing translation invariance. Multichannel CNNs. Pyramid convnet structures with pooling / strided operations. Practical CNN architecture tips (refer to existing ones). Receptive fields. Positional encoding. Popular architectures like encoder-decoder, ResNet, neural fields like SIREN and NeRF.

5. Graph Neural Networks

GNN usecases (ex: Google Maps), processing general-shaped graphs. Motivation for permutation invariance and equivalence. Images are like graphs. Message passing details (update/aggregate loop). Learning a shortest path algorithm. Graph embeddings (aggregate set of node embeddings). Unrolled GNNs. GNN extensions. Approximation power, unable to distinguish isomorphisms.

6. Neural Network Generalization

Approximation vs. Generalization. Deep nets do generalize (mostly). Generalization theory (Occam's Razor, bias-variance tradeoff, double descent). Vapnik-Chervonekis theory, which basically says that we need more data and less posssible functions to fit, but this doesn't explain deep learning. Theories behind why deep nets generalize (implicit regularization, shortest program generalizes the best).

7. Scaling Rules for Optimization

The optimization problem (minimize loss efficiently/optimally), larger width = lower learning rate, larger depth = more residual blocks, Newton's method, Gauss-Newton decomposition for optimization, proof of steepest descent under an arbitrary norm (ex: spectral norm), norms and dual norms, scaling heuristics, spectral perspective of neural nets, library of atomic modules. **(Homework 2 covered steepest descent under the spectral norm and vision architectures.)**

8. Transformers

A transformer architecture is what you should use today. Motivation (sparse/global), tokens, tokenization, attention details, self-attention, multihead self attention (MSA), transformer (ViT), transformer code, positional encoding (ex: Fourier basis), causal masking to train GPT. (Homework 3 covered transformers and other sequence models.)

9. Hacker's guide to DL

Tons of practical tips for deep learning. Split into a separate section at the top.

10. Memory and sequence modeling

CNNs for sequences. Recurrent neural networks (RNNs). Long short term memory intuition (LSTMs), modeling useful cognitive concepts in code / differentiable things. Sequence models and long memory. Teacher forcing/beam search. Larger context transformers (Transformer XL, Longformer, Big Bird, RETRO). Hypernets, code books.

11. Representation Learning I

Neural nets learn representations. Generative modeling. Neural embeddings in representation space. CLIP visualized with mappings. Visualizing CNNs. Encoders. Transfer learning (linear adaptation or finetuning). How to learn a good representation (compact, explanatory, disentangled, interpretable, make subsequent problems easy) with compression or prediction. Autoencoders via compression, clustering/k-means/VQ nets. Self-supervised learning for via prediction (works better than autoencoders) through imputation, the modern way to learn representations due to stronger training signals. Masked autoencoder (MAE), bidirectional transformers (BERT).

12. Representation Learning II: Similarity-Based

Review of first part. Similarity-based representation learning through contrastive signals. Metric learning (group similar data points in representation space). Contrastive losses, need a good distance metric. Triplet network (one positive, one negative pair) and extensions. Presenting hard negative pairs. Encoders maps onto a hypersphere. Self-supervised learning can outperform supervised pre-training sometimes. SimCLR. Metrics for alignment and uniformity. Improvements via fancy data augmentation. Larger batch sizes work better. iNaturalist case study.

13. Representation Learning Theory
Fast training of neural nets using steepest descent under the spectral norm, momentum, low precision, and UV^T computation via iteration for nanoGPT. Kernel methods, approximate functions with a bunch of “bumps”. Gaussian processes, sample a bunch of functions consistent with the data. Covariance functions. Details and proof that neural networks that are infinitely wide, with weights randomly sampled, are equivalent to Gaussian processes. Not really used in practice though. (Homework 4 covered contrastive/representation learning.)
14. Generative Models: Basics
Generative modeling is the inverse of representation learning. Formal problem specification. “Noise” is latent variables, like adjustable knobs. Can learn generators that make data directly, or learn a function that scores data quality, then generate data with high scores. Density functions, energy functions, and samplers. KL divergence. Low energy = High probability. Contrastive divergence algorithm. Autoregressive models. Diffusion models / Gaussian diffusion. Common strategy: Turn generative modeling into a sequence of supervised learning problems. Generative adversarial networks (GANs), student and teacher.
15. Generative Models and Representation Learning
Using decoders as a generative model. Variational autoencoders (VAEs) details as a Mixture of Gaussians. (GPT is great at converting LaTeX screenshots to code.) Map from an input into a Gaussian mean/variance. Tricks: Estimate integral via Monte Carlo sampling, do importance sampling, predict optimal sampling distribution with an encoder. Evidence lower-bound (ELBO). Looks like an autoencoder. Disentanglement with BigGAN. When mapping to a sphere, you will get cuts/seams in the representation.
16. Conditional Generative Models
Structured prediction (modeling joint distributions). Make it categorical by quantizing your data. Semantic segmentation (Sat2Map). Use an objective that can model structure (ex: graphical model, GAN, etc). Conditional GAN for image-to-image (ViT is the modern version of a CNN). Patch discriminator (used in previous versions of stable diffusion). Conditional VAE. Control your data via explicit inputs or latent variables. Autoregressive models are conditional generative models. Cross-attention. Text-to-image diffusion models, LLaVA image-to-text encoder. Unpaired translation, CycleGAN with cycle consistency loss. Paired data isn’t always necessary.
(Homewo**rk 5 covered various forms of autoencoders.) why crying?**
17. Out-of-distribution Generalization
Modern ML is only used in high precision, human verification systems. The world is changing, datasets do not. Data poisoning. Adversarial examples from small perturbations, even 3D-printed or in speech recognition, transfer across models trained on the same dataset. Gradient ascent to find one. Defend with adaptive data augmentation (adversarial training) by intentionally finding the worst bit of perturbation. Models will find the easiest signal to follow. Predictive features, some robust, some non-robust. Ex: Fish tend to come with a water background. Not all shortcuts are desirable. Distribution shifts, WILDS benchmark to test robustness. Distributionally robust optimization (DRO) and generalization. Class imbalance fixes. Domain adaptation.
18. Transfer Learning: Models
Pretraining for prior knowledge. Finetune some or all of the original net. Contrastive pre-training, using “glue” adapters to handle different dimensions. Domain adaptation: Learn a projection from the target domain to the source domain. Domain Adversarial Neural Networks (DANN). Unsupervised Domain Adaptation (UDA), exponential moving average. Align and Distill (ALDI). Knowledge distillation: Train a small student model to mimic a teacher model. Cross-model distillation (SoundNet), distilling an ensemble. Compression vs. distillation. Foundation models. Large cost of training models. Deep learning with no data: Just ask through prompting and few-shot learning. CLIP. Visual prompting via image inpainting works too. DALL-E 2. why crying? learn properly.. fast
19. Transfer Learning: Data
Data++: Data as a first class object to transfer knowledge about the inputs. Interpolation from a middle layer, manipulation, composition, optimization for counterfactual reasoning. Generative models as Data++ with DatasetGAN. Explaining a classifier with StylEx (GAN). Contrastive learning + generative modeling by making positive pairs through transformations in latent space. Model collapse / manifold collapse. Model-Agnostic Meta-Learning (MAML), or by sequence modeling.
20. Scaling Laws
Compute doubles every 3 months. Simple algorithms that scale well work. Power laws tend to fit scaling patterns well. The right sized model is the primary focus, exact architecture doesn’t have a

big impact (for vanilla nets). Chinchilla (DeepMind) found scaling laws to be heavily dependent on optimization methods. Guideline: For every data point you have, add one new parameter. Data manifold hypothesis to explain scaling laws. Carefully choosing non-redundant data can beat scaling laws.

21. Large Language Models

Selected concepts from 6.8611: Natural Language Processing. See the full NLP post for content.

22. AI for Musicians

Presented by Anna Huang. Diversity of musical instruments/styles. Mawf, which uses differentiable digital signal processing (DDSP). CocoNet (Counterpoint by convolutions) using masked training to generate musical scores, with annealed random sampling. Orderless Neural Autoregressive Density Estimator (OrderlessNADE). MIDI-DDSP to allow fine-grained control over generated music. Model note expression controls as scalar features, use an RNN to make pitch (sequence) with a GAN to check for realistic spectrograms (images). Generate pitch, amplitude, harmonic distribution, and noise magnitude separately. Music transformer that represents each note as an event (note on, note off, note velocity, clock advance amount). Give the transformer relative distance (positional encoding). Reinforcement learning leads to creativity. How you explore is important. Creative Adversarial Networks (CANs). Musicians use AI tools to help with brainstorming.

23. Metrized Deep Learning

A deep learning library should be like a Lego set. Steepest descent refresher. Modules made of atoms (Linear, Conv2d, Embed), bonds (ReLU, FunctionlAttention), and compounds (MLP). Goal is to predict sensitivity of output with respect to small changes in the inputs/weights. Combination/concatenation rules for modules. Scaling can hurt due to worse performance and optimal learning rate drifts. Modulo library for auto-normalization. Faster training with modular dualization (project the gradient to the weight space). This plus Newton-Schulz helped set a training speed record.

24. Inference Methods for Deep Learning

Inference definition of predicting answers for new datapoints (ML vs. statistics). Scaling of both search and learning time. Search in chess (AlphaGo). Best-of-N, beam search, chain-of-thought, verification (of code). Steering model outputs toward human preferences. New paradigm: “Just ask”. Ex: “Make it more”. In-context learning, test-time training (use test-time training on few-shot examples, or self-supervised learning), works well on ARC. STaR (Spatial-temporal Hierarchical Reinforcement Learning) by training on examples that are verified to be correct. o1 possibility: Search for CoTs that solve a problem (randomly sample), finetune on the ones that worked, and repeat.

25. Efficient Policy Optimization for LLMs

RLHF: Supervised fine tuning, preference reward model training, reinforcement learning (PPO). Reset with reference policy, constant rollin and temporary rollout. PPO++ for a better initial state distribution. Dataset reset policy optimization (DR-PO). Reduce computation/memory overhead by rewriting PPO as a regression problem (REBEL). Multi-turn interactions with a semi-realistic simulator (between two chatbots, REFUEL) to prevent performance degradation in later conversation turns.

26. Project (AGAN)

Meta learning with an online loss function (represented by a neural net with adaptable weights), applied only to the discriminator. Didn’t work due to low expressivity of the generator. GANs are very unstable. Online loss makes it even more unstable, unfortunately. Adam is much better than SGD. Training times are quite big, so a good batch size / torch.benchmark matters. Good-looking figures matter a lot for papers and blog posts. Don’t be afraid to reach out / cold email people.

Final Project

The final project will be a research project on a deep learning topic of your choice. You will run experiments and do analysis to explore your research question. You will then write up your research in the format of a blog post, which will include an explanation of the background material, the new investigations, and the results you found. You are encouraged to include plots, animations, and interactive graphics to make your findings clear.

Some examples of nice research blog posts are here: [1] [2] [3] [4].

The final project will be graded for clarity and insight as well as novelty and depth of the experiments and analysis.

Project Details

You will complete a research project on a scientific question related to the topics covered in this course and will write up your findings and insights in the form of a blog, with an emphasis on clear communication. Write the blog you would like to read and would learn something from.

Collaboration

You can work in groups of up to three. Group projects require work proportional to the size of the group (details below).

What we expect

Your project should investigate a scientific question and present the resulting analysis in the format of a blog. Despite being a blog, the writing should be technical, of high quality, and not overly informal. What do we mean by technical writing? Each claim you make should be precise and supported. You can support a claim either by a citation, an experiment, or a mathematical argument. B**e sparing with sentences that are subjective or arguable, and only include them with proper context. The course projects should go beyond re-implementing existing methods and instead seek new insights and understanding of topics discussed in class. It is not enough to do a literature review, nor to directly re-implement a method without making any changes. Projects can explore the theory or applications of deep learning. Application-oriented projects should ask questions that further our understanding of deep learning and not just your application area.**

For example, here are two possible applied projects, a mediocre one and a good one:

1. **"Drug classification with deep nets"** Apply deep nets to classifying molecules by their toxicity. Show that transformers get higher accuracy than RNNs on this problem. Achieve a high accuracy on the test set. [mediocre]
2. **"Adaptive computation time transformers for molecular analysis"** Identify that different molecules come in wildly different sizes, and should require different amounts of computation to analyze. Show how to modify a transformer to have depth or width that is adaptive to the size of the query molecule. Analyze how your system trades off time/memory for accuracy, and how properties of the molecule graphs affect this tradeoff. [good]

Length

Your blog should be around 2000–3000 words and contain 4–6 figures/tables visualizing aspects of your study (the word count is not including the tables and figures). For groups of size 2 we will expect 1.5× as much work and for groups of size 3 we will expect 2× the work. This does not mean 2× the length, but rather 2× the depth of content (a rough measure of depth could be, but does not have to be, the number of experiments you run or the number of hypotheses you investigate). Pragmatically, each group member should be spending 4 weeks worth of work on the project (the last 4 weeks of the class), which equates to about 36 hours of work per member.

Projects related to your outside work

You are welcome to work on a project that is related to your thesis topic or other outside interests. However, the work you do for this class should be novel to this class. You should not submit work for which you have gotten, or will get, credit toward another class or degree program. Of course it's fine, and wonderful, if after this class you extend your project into a publication or blog post that you can share with the world.

Evaluation

The blog will be graded by novelty, quality, and clarity of the content. The grade will be determined by how well your blog offers fresh insights and in-depth analysis.

Timeline

1. Submit proposal [10% of grade]: Submit a proposal as a one-page pdf. Provide an outline of your plan for the project and questions you will investigate / analysis you'll conduct in the course of it. It may help to define a set of hypotheses you will test. An integral aspect of the proposal is to define a project idea that is both realistic and ambitious in scope. We recommend that you use the project proposal stage to get feedback from the teaching staff on the project's feasibility and whether the proposal satisfies the project expectations of the class.
2. Submit blog [90% of grade]: See blog submission format in the section below.

Project Ideas

A list of possible project ideas is available. Please note that these are guiding ideas and that you are encouraged to develop your own ideas outside of this list.

Grading Rubric
See the grading rubric.
The project will be evaluated on the following criteria: novelty (20 points), technical soundness and content (30 points), clarity (20 points), literature review (20 points), and formatting (10 points).
Formatting
You should submit a single .zip file which includes an index.html. When we open index.html in a standard browser (Chrome), it should render the blog without requiring an internet connection (no dependencies on any files other than those in the.zip). We provide a basic template you may use here: Blog Template (ZIP). But you don't have to use this. Feel free to get creative with the look and format of your blog.
FAQs
- No late submissions will be accepted: In order to meet the grading deadlines, we will not be able to accept late submissions for the final project.
- You can change your final project from the proposal (the grade will not depend on similarity to the proposal), at the risk of not getting feedback from the proposal stage. However, you can come to office hours to discuss with course staff.
- This year we will not publish your submitted blogs, so you can consider that they will be kept private amongst the course staff. You are welcome to self-publish your finished projects. If you would like to see examples, see the blog posts from 2023. Other high quality blog-format posts are viewable at Distill, although we acknowledge that Distill sets a high standard.

Final Project Grading Rubric

Novelty: 20 pts
The project is innovative / creative / interesting / tackling a fundamental research question. Examples of this could include any of the following: a) exposing a previously unknown connection between two different methods, b) exploring a component of current methods that is not well understood, c) proposing an improvement to an existing method, or d) interpreting or analyzing the learned mechanisms and representations in an existing model.
The project is a creative/important application of deep learning that provides novel scientific insights into deep learning. Applying an existing method to a new dataset and reporting results is insufficient. Examples of ways to achieve sufficient novelty: a) exposing fundamental limitations of current deep learning methods and experimentally or theoretically analyzing why the method is underperforming, b) exploring how additional structure or knowledge from an application area can be built into deep learning methods to improve results.
Technical Soundness and Content: 30 pts
The methods are sound, the math is correct, and conclusions are justified by the results.
The experiments/analyses are well designed to test the scientific claims. Uncertainty is property quantified.
The project is well scoped for the course (i.e., not too ambitious to be finished within the timeframe and with available resources).
Clarity: 20 pts
The project proposal is well written and structured, and is made understandable to the reader. The figures are clear and help the reader to understand your analysis.
Is the motivation clear? Is there a discussion of implications and limitations? Is there a clear conclusion based on the experiments in the paper?
Literature Review: 20 pts
Is the proposed project well motivated by a gap in prior research? Please include citations to prior work/papers to indicate the gap in previous literature that your project addresses.
Formatting: 10 pts
You should include the following content:
1. An introduction or motivation
2. Background and related work with literature cited
3. A description of your methods and experiments with figures showing the method or setup
4. An analysis of the results of your experiments with figures showing the results
5. A conclusion or discussion, with mention of limitations